Lab Practice Problem 6

Filename: `practice6.c`

In this activity, you will implement the **selection sort algorithm** to arrange the elements of an array in ascending order. You will practice working with one-dimensional arrays, passing arrays to functions, and organizing a program into smaller, reusable functions.

Try It Out

Write a program that sorts an array of integers in ascending order. The algorithm works by repeatedly finding the smallest element in the unsorted portion of the array and moving it to its correct position. After each pass, one additional element is placed in its final sorted position.

Example

Consider the following sequence:

50 20 10 30 40

During the first pass, the smallest element, 10, is exchanged with the element at index 0:

10 20 50 30 40

During the second pass, 20 is already the smallest element in the remaining unsorted portion, so no exchange is necessary. The process continues until the sequence becomes:

10 20 30 40 50

In `main()`, declare an array with a capacity of 100 integers, initialize its first five elements, and use `n` to track the number of elements being processed:

```
int nums[100] = {50, 20, 10, 30, 40};
int n = 5;
```

To complete the program, implement the following functions using the specified prototypes:

`void print_array(const int arr[], int size)`
Displays the first `size` elements of the array, separated by spaces, followed by a newline.

`int find_min_index(const int arr[], int start, int size)`
Returns the index of the smallest element between `start` and `size-1`, inclusive.

`void swap(int arr[], int i, int j)`
Exchanges the values stored at indices `i` and `j` in the array.

`void selection_sort(int arr[], int size)`
Sorts the first `size` elements of the array in ascending order using the selection sort algorithm.

Important: Use the specified function signatures. Do not use global variables.

Input

No user input is required. Use the predefined values in the array declared in `main()`.

Output

Display the first `n` elements before and after sorting. Print each sequence on a separate line, with elements separated by spaces.

Sample Output

```
50 20 10 30 40
10 20 30 40 50
```

Guide Questions

1. The array `nums` has a capacity of 100 elements, but `n` is `5`. What is the difference between these two values? What would happen if `print_array(nums, 100)` were called?
2. The variable `n` in `main()` is passed to functions with a parameter named `size`. Why does this work even though the names are different?
3. Why is `const` used in the array parameters of `print_array()` and `find_min_index()`, but not in `swap()` and `selection_sort()`?
4. Given the initial array values, what will the following function calls return? Explain your answers.

   ```
   find_min_index(nums, 0, 5)
   find_min_index(nums, 2, 5)
   ```

5. Why do changes made to the array inside `swap()` also affect the original array in `main()`?
6. After each pass of selection sort, what can you conclude about the elements at the beginning of the array?
7. Suppose the array contains duplicate values. How would this affect the behavior of `find_min_index()` and the final result of `selection_sort()`?

Sample Solution

```c
#include <stdio.h>

// Function Prototypes
void print_array(const int arr[], int size);
int find_min_index(const int arr[], int start, int size);
void swap(int arr[], int i, int j);
void selection_sort(int arr[], int size);


int main(void) {
    // Do not introduce int nums[n] to students!
    int nums[100] = {50, 20, 10, 30, 40};
    int n = 5;

    // Display the original array
    print_array(nums, n);

    // Sort the array in ascending order
    selection_sort(nums, n);

    // Display the array after sorting
    print_array(nums, n);

    return 0;
}


// Function Definitions
void print_array(const int arr[], int size) {
    // Display all elements on one line, separated by spaces
    for(int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int find_min_index(const int arr[], int start, int size) {
    // Assume the first element in the search range is the smallest
    int min_idx = start;

    // Search the remaining elements for a smaller value
    for(int i = start+1; i < size; i++) {
        // Update the index when a smaller element is found
        if( arr[i] < arr[min_idx] ) {
            min_idx = i;
        }
    }

    // Return the index of the smallest element
    return min_idx;
}

void swap(int arr[], int i, int j) {
```

```
54      // Exchange the values at the specified indices
55      int tmp = arr[i];
56      arr[i] = arr[j];
57      arr[j] = tmp;
58  }
59
60  void selection_sort(int arr[], int size) {
61      // Move the smallest remaining element into position
62      // during each pass through the array
63      for(int i = 0; i < size-1; i++) {
64          // Find the smallest element in the unsorted portion
65          int min_idx = find_min_index( arr, i, size );
66
67          // Swap only if the smallest element is not already
68          // in its correct position
69          if( min_idx != i ) {
70              swap( arr, i, min_idx );
71          }
72      }
73  }
```